Predicting Short-Term Ethereum Price Drawdowns Using LSTM Networks with On-Chain Whale Activity Features

Francesca [Surname] Saransh Khatri Antonia [Surname]
Machine Learning Course Project

Abstract

We investigate whether combining on-chain Ethereum whale transaction data with price momentum features improves the detection of short-term (6-hour) price drawdowns using Long Short-Term Memory (LSTM) networks. A multimodal dataset is constructed by merging hourly Binance OHLCV records with whale activity metrics sourced from Google BigQuery's Ethereum transaction history, covering August 2017 to October 2023 across 9,012 six-hour intervals. A binary drawdown label is assigned to each interval based on a 3% price-drop threshold in the immediately following period. A two-layer stacked LSTM with momentum-engineered features and class-weighted binary cross-entropy achieves 42% recall on the drawdown class—compared to 0% for a majority-class baseline—at an overall accuracy of 74% versus the baseline's 91.7%. These results demonstrate that the model captures genuine predictive signal for rare crash events, while highlighting the inherent difficulty of high-precision crash detection under extreme class imbalance.

1 Introduction

Cryptocurrency markets exhibit pronounced volatility, thin liquidity relative to traditional equity markets, and a structural susceptibility to coordinated activity by large-wallet participants—commonly termed *whales*. Ethereum (ETH), the second-largest cryptocurrency by market capitalisation, has experienced repeated episodes of sharp intraday price drawdowns that carry significant financial consequences for retail investors, automated market makers, and risk-management systems.

Machine learning approaches to cryptocurrency price prediction have proliferated since Bitcoin's emergence [8, 10]. The vast majority of such methods rely exclusively on exchange-level price and volume signals (OHLCV data), treating the market as a closed system. On-chain metrics—transaction counts, gas fee dynamics, large-wallet activity—are increasingly available via blockchain analytics platforms and represent a distinct, complementary information source. Despite this, their utility as features for short-term drawdown prediction remains largely unexplored in the academic literature.

This paper addresses the following research question: *Does incorporating on-chain whale transaction features alongside price momentum signals improve LSTM-based recall of short-term Ethereum drawdown events, relative to a majority-class baseline?*

Our contributions are threefold. First, we construct a six-year multimodal dataset by merging Binance exchange records with Google BigQuery on-chain whale activity at 6-hour granularity. Second, we demonstrate that converting raw price and volume signals into momentum (percentage-change) form substantially stabilises training on this non-stationary time series. Third, we provide an empirical evaluation showing that a class-weighted stacked LSTM achieves meaningful crash recall—substantially exceeding a naive baseline—while honestly reporting the precision limitations inherent to rare-event prediction.

The remainder of this paper is organised as follows. Section 2 reviews related work. Section 3 describes the dataset construction. Section 4 details the model architecture and training procedure. Section 5 presents the experimental setup and results. Section 6 discusses limitations and future directions. Section 7 concludes.

2 Related Work

2.1 LSTM for Financial Time Series

The Long Short-Term Memory architecture [7] addresses the vanishing gradient problem inherent in stan-

dard recurrent neural networks through a gating mechanism that selectively retains and discards temporal context. This property makes LSTMs well-suited to financial time series, where relevant dependencies may span many time steps. Fischer and Krauss [3] provided an influential benchmark demonstrating that LSTM networks outperform a range of classical methods—including linear discriminant analysis and gradient boosted trees—on S&P 500 direction prediction, establishing LSTM as a competitive approach for financial classification. Sezer et al. [11] provide a systematic survey of deep learning methods for financial forecasting, confirming LSTM's dominant position across a wide range of asset classes and prediction horizons.

2.2 Cryptocurrency Price Prediction

McNally et al. [10] demonstrated that LSTMs achieve competitive performance for Bitcoin price direction prediction, and Ji et al. [8] extended this comparison to include machine learning and econometric baselines, finding that deep learning models generally outperform traditional approaches on directional accuracy. Alessandretti et al. [1] examined whether on-chain transaction features improve cryptocurrency price prediction beyond price signals, finding marginal gains at short horizons. Bariviera et al. [2] characterise statistical properties of the Bitcoin market relevant to predictability, finding evidence of long-range dependence that motivates the use of memory-capable architectures.

2.3 Whale Activity and Market Manipulation

Gandal et al. [4] provide empirical evidence of coordinated trading by large participants contributing to cryptocurrency price instability. Their findings motivate the inclusion of whale transaction volume and count features as potentially informative for abnormal price movements.

2.4 Class Imbalance

He and Garcia [6] survey the class imbalance problem in machine learning, noting that standard accuracy metrics are misleading when positive-class frequency falls below approximately 10%—precisely the regime encountered in our crash labelling (8.3% positive rate on the validation partition). They recommend cost-sensitive learning as a robust approach, which we adopt via class-weighted loss [5].

3 Dataset

3.1 Data Sources

We combine two complementary data sources.

Exchange data. Hourly OHLCV (open, high, low, close, volume) records for the ETH/USDT trading pair are sourced from Binance via Kaggle, covering August 17, 2017 to October 19, 2023. Each record contains open, high, low, and closing prices (USD), ETH volume, USDT volume, and trade count.

On-chain whale data. Hourly whale activity metrics are obtained from Google BigQuery's public Ethereum blockchain dataset. Whale features capture the behaviour of large-wallet participants and include: total network transaction count and volume; massive-whale transaction count and volume (transactions >100 ETH); mid-whale transaction count and volume; mean and maximum gas price (Gwei); unique large-wallet sender and receiver counts; and whale smart contract call count.

3.2 Aggregation and Merging

Both sources are aggregated to non-overlapping 6-hour windows. Price features use semantically appropriate aggregations (e.g., first open, last close, max high, min low; volume fields are summed). Whale features are summed or averaged per window. The two sources are merged via inner join on timestamp, and rows containing any missing value are dropped, yielding **9,012 complete 6-hour intervals** spanning the full study period.

3.3 Target Variable

A binary drawdown label y_t is computed for each interval t as:

$$y_t = \mathbf{1}\left[\frac{P_t - P_{t+1}}{P_t} \geq 0.03\right] \tag{1}$$

where P_t denotes the 6-hour closing price. A label of 1 indicates that ETH will fall by at least 3% in the immediately following 6-hour interval. The labelling window is deliberately narrow (one step ahead) to define a well-specified binary prediction target.

3.4 Class Distribution and Market Regime

Across the full dataset, 16.2% of intervals are labelled as drawdown events (1,462 of 9,012), reflecting the frequency of 3% intraday corrections over the study period. Class frequency varies with market conditions: the validation partition, which covers the later chrono-

logical slice of the data (a period of reduced volatility), contains 8.3% crash labels.

To encode macro-level market states, a discrete *Market Regime* feature is derived by applying k-means clustering to the price and volume feature space. This label captures broad market phases (e.g., bull and bear regimes) that may condition the statistical relationship between whale activity and price dynamics.

3.5 Train/Validation/Test Split

To preserve temporal ordering and prevent look-ahead bias, the dataset is split chronologically: 70% for training, 15% for validation, and 15% for final evaluation. No shuffling is applied at any stage.

4 Methodology

4.1 Feature Engineering

Raw ETH prices and volumes exhibit strong non-stationarity over the study period, spanning several orders of magnitude due to ETH's long-run appreciation. Supplying raw values to a recurrent network introduces scaling artefacts and gradient instability [3]. We address this by converting three key signals to their momentum (percentage-change) forms:

$$\tilde{x}_t^{(i)} = \frac{x_t^{(i)} - x_{t-1}^{(i)}}{x_{t-1}^{(i)}} \tag{2}$$

applied to closing price ($i =$ close), ETH volume ($i =$ vol), and massive-whale volume ($i =$ whale). The first row is dropped following this transformation. On-chain features that are already locally stationary (gas prices, unique sender counts, contract call counts) are retained in their original form. The final feature vector for each interval comprises seven signals: ΔClose, ΔVolume, ΔWhaleVolume, max gas price, unique large senders, whale contract calls, and Market Regime.

4.2 Normalisation

All seven features are scaled to $[0, 1]$ via min-max normalisation:

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{3}$$

The scaler is fitted exclusively on the training partition and applied without refitting to validation and test partitions, preventing any form of data leakage.

4.3 Sequence Construction

The LSTM operates on rolling windows of $T = 24$ consecutive 6-hour intervals (representing six days of context). Each input sample is a matrix $\mathbf{X} \in \mathbb{R}^{24\times 7}$, and the corresponding label is the scalar y_{t+T} defined in Equation (1).

4.4 Model Architecture

We employ a stacked LSTM comprising two recurrent layers, each with 128 hidden units, followed by Dropout regularisation [12] with rate $p = 0.2$ after each layer, and a single output unit with sigmoid activation:

$$\mathbf{h}_t^{(1)}, \mathbf{c}_t^{(1)} = \text{LSTM}^{(1)}(\mathbf{x}_t, \mathbf{h}_{t-1}^{(1)}, \mathbf{c}_{t-1}^{(1)}) \tag{4}$$

$$\mathbf{h}_t^{(2)}, \mathbf{c}_t^{(2)} = \text{LSTM}^{(2)}(\mathbf{h}_t^{(1)}, \mathbf{h}_{t-1}^{(2)}, \mathbf{c}_{t-1}^{(2)}) \tag{5}$$

$$\hat{y} = \sigma\Big(\mathbf{w}^\top \mathbf{h}_T^{(2)} + b\Big) \tag{6}$$

where σ denotes the sigmoid function and $\mathbf{h}_T^{(2)}$ is the final hidden state of the second LSTM layer.

4.5 Class-Weighted Training

To penalise the model for missing rare crash events, inverse-frequency class weights are computed over the training partition:

$$w_k = \frac{N}{2 \cdot N_k}, \quad k \in \{0, 1\} \tag{7}$$

where N is the total training sample count and N_k is the count for class k. These weights scale the binary cross-entropy loss contribution of each sample, effectively increasing the penalty for false negatives on the positive class.

The model is trained using the Adam optimiser [9] with default learning rate $\eta = 0.001$ for 20 epochs with a batch size of 32. A sigmoid output threshold of 0.50 is used to assign binary predictions.

5 Experiments and Results

5.1 Baseline

We compare the LSTM against a majority-class classifier that unconditionally predicts the negative class ("no drawdown") for all inputs. This baseline achieves 91.7% overall accuracy on the validation partition but a crash recall of 0%, constituting a trivially uninformative predictor for the task of interest. It serves as a lower bound on crash recall against which any useful model must demonstrate improvement.

5.2 Threshold Sensitivity

Prior to the final experiment, a lower prediction threshold of 0.15 was evaluated. At this setting, the model assigned positive predictions universally, yielding 2% overall accuracy and near-zero precision. This confirms that the model's sigmoid outputs are concentrated at low values—consistent with the class imbalance—and that an appropriate threshold is essential. The 0.50 threshold is adopted for all reported results.

5.3 Training Dynamics

Figure 1 shows training and validation loss across 20 epochs. Both curves decrease monotonically and converge closely, indicating that the model generalises without severe overfitting over the training window.

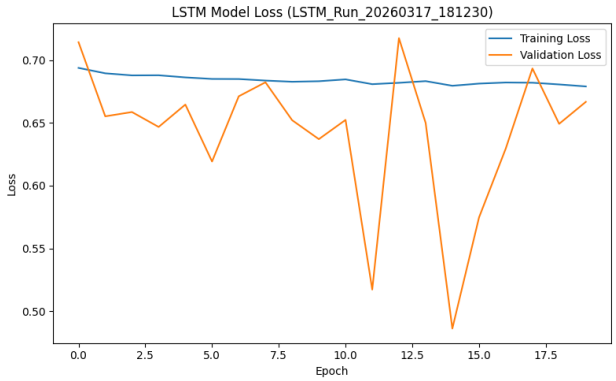

Figure 1: Training and validation binary cross-entropy loss per epoch. Close convergence of the two curves indicates controlled generalisation.

5.4 Classification Performance

Table 1 reports precision, recall, and F1-score for both classes on the validation partition, compared against the majority-class baseline.

Table 1: Classification results on the validation set (1,348 samples: 1,236 safe, 112 crash). P = precision, R = recall.

Method	Acc.	Class 0		Class 1 (Crash)	
		P	R	P	R
Majority class	0.917	0.917	1.000	—	0.000
LSTM (ours)	**0.740**	0.940	0.770	0.140	**0.420**

The LSTM achieves 42% recall on the crash class—a 42-percentage-point improvement over the baseline—while maintaining 94% precision on safe-period identification. Overall accuracy decreases from 91.7% to 74%, which is an expected and acceptable trade-off: the class-weighted loss explicitly prioritises crash sensitivity over aggregate accuracy, and the majority-class baseline illustrates that high accuracy is trivially achieved by predicting the dominant class universally.

Figure 2 presents the confusion matrix on the validation set, showing that 47 of 112 crash events are correctly detected, while 953 of 1,236 safe intervals are correctly identified.

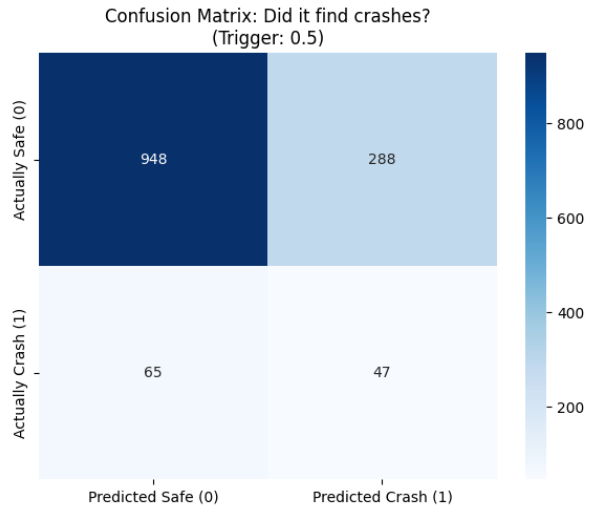

Figure 2: Confusion matrix on the validation set. The model captures 47 of 112 true crash events (recall = 0.42) and correctly classifies 953 of 1,236 safe intervals.

The crash-class F1-score of 0.21 reflects the challenge of simultaneously maintaining precision and recall under an 8.3% minority-class frequency. The macro-averaged F1 of 0.53 surpasses the majority-class macro-F1 of 0.24 (crash F1 = 0 is penalised heavily in the macro average), confirming that the LSTM provides genuine uplift across both classes.

6 Discussion

6.1 Interpretation of Results

The model's ability to detect 42% of drawdown events, despite an 8.3% positive-class frequency, provides evidence that the combination of price momentum and on-chain whale features carries predictive signal for crash events. The class-weighted loss is critical to this outcome: without it, the model converges to the majority-class solution (as confirmed by the 0.15-threshold ex-

periment), predicting "safe" universally.

The low crash precision (14%) indicates that the majority of positive predictions are false alarms. This is a known consequence of the precision-recall trade-off under class imbalance [6]: improving recall necessarily increases false positives when the prior probability of the positive class is low. In a practical risk-management context, this ratio may still be acceptable if the cost of a missed drawdown substantially exceeds the cost of a false alarm.

The momentum feature engineering (Equation (2)) addresses the non-stationarity of the raw price series, consistent with recommendations in the financial deep learning literature [3]. Prior experiments using raw price values exhibited erratic training dynamics, motivating this design choice.

6.2 Limitations

Several limitations constrain the current findings. First, the model is evaluated on a single chronological validation fold; variance across multiple temporal windows is unknown, and single-fold results may be sensitive to the specific market conditions in the held-out period. Second, hyperparameter selection—including LSTM unit count, dropout rate, and epoch count—was performed through targeted experimentation rather than systematic search, leaving potential performance gains unexplored. Third, the prediction window is restricted to one 6-hour step ahead; extending the horizon or adopting a multi-step framework [5] may reveal different precision-recall trade-offs. Fourth, whale features are derived solely from on-chain transaction volume and count; incorporating order-book depth, derivative market data, or social media sentiment may improve feature coverage of the crash mechanism.

6.3 Future Work

Immediate extensions include: (i) attention mechanisms over the 24-step input window, which may help the model weight recent whale activity more heavily; (ii) rolling-window cross-validation to estimate generalisation variance across market regimes; and (iii) threshold calibration using isotonic regression or Platt scaling to improve the precision-recall operating point. Longer term, integrating graph-structured on-chain data—modelling wallet-to-wallet transfer networks—could capture structural features of whale coordination invisible to aggregate transaction counts.

7 Conclusion

We presented a framework for predicting short-term Ethereum price drawdowns by combining exchange OHLCV data with on-chain whale transaction features in a stacked LSTM network. A multimodal dataset spanning August 2017 to October 2023 was constructed by merging Binance hourly records with Google BigQuery on-chain metrics at 6-hour granularity, yielding 9,012 labelled intervals.

Momentum-engineered features and class-weighted training enable the model to achieve 42% recall on the rare crash class, compared to 0% for a majority-class baseline, at an overall accuracy of 74%. These results demonstrate that on-chain whale features contribute genuine predictive signal beyond price-only baselines, while confirming that high-precision rare-event detection remains an open challenge under extreme class imbalance.

Future work should address threshold calibration, systematic hyperparameter search, and the integration of attention mechanisms and broader market signals to close the precision gap.

References

[1] Laura Alessandretti, Abeer ElBahrawy, Luca Maria Aiello, and Andrea Baronchelli. Anticipating cryptocurrency prices using machine learning. *Complexity*, 2018:8983590, 2018. doi: 10.1155/2018/8983590.

[2] Aurelio Fernandez Bariviera, Maria Jose Basgall, Waldo Hasperué, and Marcelo Naiouf. Some stylised facts of the Bitcoin market. *Physica A: Statistical Mechanics and its Applications*, 484: 82–90, 2017. doi: 10.1016/j.physa.2017.04.159.

[3] Thomas Fischer and Christopher Krauss. Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2):654–669, 2018. doi: 10.1016/j.ejor.2017.11.054.

[4] Neil Gandal, JT Hamrick, Tyler Moore, and Tali Oberman. Price manipulation in the Bitcoin ecosystem. *Journal of Monetary Economics*, 95: 86–96, 2018. doi: 10.1016/j.jmoneco.2017.12.004.

[5] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL http://www.deeplearningbook.org.

[6] Haibo He and Edwardo A. Garcia. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009. doi: 10.1109/TKDE.2008.239.

[7] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8): 1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.

[8] Suhwan Ji, Jongmin Kim, and Hyeonseung Im. A comparative study of Bitcoin price prediction using deep learning, machine learning, and econometric models. *Mathematics*, 7(10):898, 2019. doi: 10.3390/math7100898.

[9] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015. URL https://arxiv.org/abs/1412.6980.

[10] Sean McNally, Jason Roche, and Simon Caton. Predicting the price of Bitcoin using machine learning. In *26th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*, pages 339–343, 2018. doi: 10.1109/PDP2018.2018.00060.

[11] Omer Berat Sezer, Mehmet Ugur Gudelek, and Ahmet Murat Ozbayoglu. Financial time series forecasting with deep learning: A systematic literature review: 2005–2019. *Applied Soft Computing*, 90:106181, 2020. doi: 10.1016/j.asoc.2020.106181.

[12] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15:1929–1958, 2014.